

SPRINT 4 COMPLETION REPORT

SHADOW OS v2 — Live Memory Integration

' COMPLETE — 417/417 TESTS

Date: 2026-07-11 · Sprint 4 · FOREX ENGINE PRO

Executive Summary

Sprint 4 wires the manager tier built in Sprints 1–3 (RuntimeDomainManager, TradeIntentManager, MemoryManager) into the running Live Engine (telemetry/server.js). The system now **remembers across restarts** : every

boot validates memory, recovers from the newest *valid* snapshot, records its own recovery, and every trade open/close/restart is persisted as a permanent, idempotent memory event. Every shutdown — clean or violent — is survivable: SIGTERM flushes and snapshots under a hard 5-second deadline; SIGKILL is absorbed by session-scoped locks and dedupe-keyed writes.

This is the first sprint that touches telemetry/server.js. The integration

is **additive and flag-gated**

(SHADOW_OS_MEMORY, default on): with the flag

off, the server byte-for-byte reproduces its pre-Sprint-4 behavior, including instant death on signals. Every hook is best-effort try/catch — ****a memory failure can never block trading****. index.js remains FROZEN, untouched.

Deliverables

1. LiveMemoryIntegration (telemetry/managers/LiveMemoryIntegration.js)

The bridge module. One instance per server process.

Startup recovery pipeline (recoverOnStartup):

1. Acquire pg session-scoped advisory lock (LOCK_CLASS=21320, LOCK_OBJ=20307) on a dedicated client — duplicate startup degrades to observe-only mode, it never fights over state
2. Validate memory (MM.validateMemory) — structural corruption is **quarantined to CORRUPTED, never deleted**
3. Find the newest snapshot that passes integrity validation (per-domain checksum + history-row existence), walking back over invalid snapshots (up to 20 candidates) — invalid snapshots are ****skipped and logged, never deleted****
4. Recover runtime domains, open trade intents (CREATED/VALIDATED/APPROVED), and the memory summary
5. Detect drift between the replay-built live state and the v2 live domain — logged to consistency_log, observe-only
6. Write a dedupe-keyed SYSTEM_RECOVERY event and take a post_recovery snapshot

Runtime hooks: recordTradeOpen / recordTradeClose /

recordBotRestart — idempotent via dedupe_key (minute-bucket for trades, per-boot for lifecycle events), fire-and-forget tracked for flush.

Persistence & shutdown: periodic snapshot + memory summary

(SHADOW_OS_PERSIST_MS, default 5 min); gracefulShutdown = flush in-flight

writes (allSettled + timeout) !' SYSTEM_SHUTDOWN event !' final snapshot !'

lock release.

Degradation ladder: init failure !' inert no-op; lock held !' observe-only;

hook failure !' logged, counted, trading continues.

2. server.js Integration (flag-gated, additive)

Hook point	What it does
Startup (after restoreLiveState())	init() !' recoverOnStartup({liveState}) !' startPeriodicPersistence()
Trade open (openM stdout branch)	recordTradeOpen (best-effort)
Trade close (exit-block parser)	recordTradeClose (best-effort)
Bot restart loop	recordBotRestart; restart suppressed during shutdown
SIGTERM / SIGINT	kill bot first !' 4s memory shutdown budget !' hard 5s unref'd exit deadline
GET /api/memory-integration/status	live counters, boot id, lock state

With SHADOW_OS_MEMORY=off: no init, no hooks, **no signal handlers** — the

pre-Sprint-4 instant-death-on-redeploy behavior is fully preserved.

3. Managers Barrel (telemetry/managers/index.js)

Now exports LiveMemoryIntegration + LOCK_CLASS, LOCK_OBJ, OPEN_INTENT_STATUSES, SNAPSHOT_WALKBACK_LIMIT.

4. Test Drivers (telemetry/tests/drivers/)

Real second-OS-process actors (never server.js — that would spawn the live

bot): mi_hold_lock.js (holds the advisory lock until killed),

mi_recover_once.js (full recovery + shutdown, JSON report),

mi_crash_holder.js (acquires lock, writes mid-flight, spins for SIGKILL).

Test Results

Sprint 4 Tests (21 new, all passing)

Suite	Tests	Pass	Fail
Integration (mi_integration.test.js)	18	18	0
Cross-process stress (mi_process.test.js)	3	3	0

TOTAL	21	21	0
--------------	-----------	-----------	----------

Scenario coverage (the 10 required categories):

#	Scenario	Verified by
1	Normal restart	shutdown !' new boot recovers from shutdown snapshot
2	Railway redeploy simulation	graceful shutdown flush + next-boot recovery over a corrupt snapshot
3	Crash recovery	SIGKILL'd holder !' next boot recovers cleanly
4	Power loss (no flush)	SIGKILL mid-write; awaited writes durable, lock auto-freed
5	Duplicate startup protection	second OS process AND in-process instance refused, observe-only
6	Concurrent recovery	two racing processes !' exactly one wins the lock
7	Snapshot validation	valid passes; tampered checksum + missing history fail
8	Memory corruption handling	structural corruption quarantined (CORRUPTED), never deleted
9	Recovery timing	< 10s asserted (observed: 37–293ms)
10	Large history recovery	2 000 bulk memories; recovery < 15s (observed: < 1s)

Plus: idempotent trade/restart/recovery/shutdown writes, drift detection

logging, in-flight write flush on shutdown, kill-switch no-op safety,

degraded-init safety.

Full Regression (zero failures)

Sprint	Suite	Tests	Pass	Fail
0	Schema + smoke	19	19	0
1	RDM (unit/integration/simulation/stress)	107	107	0
2	TIM (unit/integration/×RDM/simulation/stress)	169	169	0
3	MM (unit/integration/×RDM×TIM/persistence)	87	87	0
3	MM stress (isolated run)	14	14	0
4	MI integration + cross-process	21	21	0
	TOTAL	417	417	0

Note: multi-file runs require --test-concurrency=1 (mm_persistence uses pg_terminate_backend, which kills concurrent suites' connections). The smoke.test.js file-level timeout after all tests pass is a pre-existing Sprint 0 artifact (db-adapter pool never closes).

Gate Criteria

Gate	Criterion	Pass Condition	Status
GATE-4. A	State survives restart	New boot recovers domains/intents/memory from last valid snapshot	' PASS
GATE-4. B	Kill switch	SHADOW_OS_MEMORY=off !' zero behavior change, no signal handlers	' PASS

GATE-4.C	Never blocks trading	All hooks best-effort; degraded init = inert no-op	' PASS
GATE-4.D	Duplicate startup protection	Second process/instance refused via advisory lock; observe-only	' PASS
GATE-4.E	Idempotency	All writes dedupe-keyed; re-runs never double-write	' PASS
GATE-4.F	Sacred Constraint	Corruption quarantined/skipped — nothing ever deleted	' PASS
GATE-4.G	Bounded shutdown	Flush + snapshot under hard 5s exit deadline (unref'd)	' PASS
GATE-4.H	Zero regressions	396/396 baseline tests still passing	' PASS
GATE-4.I	index.js untouched	git diff on index.js = 0 lines	' PASS

Architecture Decisions of Record

1. Observe-only live object

— Sprint 4 never mutates the server's live state from recovered data; drift is logged, not corrected. The v2 store becomes authoritative in a later sprint.

2. Dedicated lock client

— the advisory lock lives on its own pg client, never a pooled one, so pool churn cannot silently drop the lock.

3. Session-scoped (not transaction-scoped) advisory lock

— survives between queries, freed automatically by Postgres on ANY process death, including SIGKILL. No stale-lock cleanup logic needed.

4. Flush = allSettled + timeout

— shutdown never awaits a hung write forever; the hard exit deadline is unref'd so it can never keep the process alive.

5. Walk-back, never repair

— an invalid snapshot is evidence, not garbage. Recovery skips it and logs; a human decides its fate.

Known Limitations

- Recovered v2 state is not yet consumed by trading logic (observe-only by design — see Decision 1).
- Periodic persistence interval is coarse (5 min default); trade events are written immediately, so the exposure window applies to snapshots only.
- Redeploy lock race (Sprint 5 item):** Railway overlaps old and new processes on redeploy. The advisory lock is attempted exactly once at

startup — if the new process boots inside the old one's "d5s shutdown window, it stays observe-only (no persistence) until the next restart.

Fails safe (trading unaffected), diagnosable via

GET /api/memory-integration/status and the recovery:duplicate_startup

WARN. Fix planned for Sprint 5: bounded lock retry with backoff.

- **Minute-bucket dedupe can drop rapid repeats:** two closes of the same symbol within the same minute record only the first memory event (stdout parsing has no trade id). The events table and ShadowM remain the authoritative trade record; include a signal id in the dedupe key when the parser can supply one.
- smoke.test.js pool-hang artifact inherited from Sprint 0 (documented, harmless).

Sprint Status

' **SPRINT 4 PASSED**— 417/417 tests, 9/9 gates, 0 rows deleted, index.js untouched. The migration program pauses here by instruction: no Sprint 5 work has been started. Next planned: KnowledgeManager (Sprint 5), RecoveryManager + ValidationManager (Sprint 6).

FOREX ENGINE PRO | Sprint 4 Report v1.0 | SHADOW OS v2 Live Memory Integration | 2026-07-11